

Olist E-Commerce Database — RAG Context

Purpose: This document provides the complete schema, relationships, and business logic for the Olist e-commerce database. Use this context to generate accurate, secure SQL queries from natural language.

1. Database Schema

users

Customer information and location data.

Column	Type	Key	Description
customer_id	VARCHAR(50)	PK	Unique ID per order (can differ for same person)
customer_unique_id	VARCHAR(50)		Actual person ID (consistent across orders)
customer_zip_code_prefix	INT		Postal code prefix
customer_city	VARCHAR(100)		Customer's city
customer_state	VARCHAR(20)		Customer's state (e.g., SP, RJ)

```
CREATE TABLE users (  
  customer_id VARCHAR(50) PRIMARY KEY,  
  customer_unique_id VARCHAR(50),  
  customer_zip_code_prefix INT,  
  customer_city VARCHAR(100),  
  customer_state VARCHAR(20)  
);
```

products

Product catalog with dimensions and categories.

Column	Type	Key	Description
product_id	VARCHAR(50)	PK	Unique product identifier
product_category_name	VARCHAR(100)		Product category (in Portuguese)
product_name_lenght	INT		Name length (characters)
product_description_lenght	INT		Description length (characters)

Column	Type	Key	Description
product_photos_qty	INT		Number of photos
product_weight_g	INT		Weight in grams
product_length_cm	INT		Length in cm
product_height_cm	INT		Height in cm
product_width_cm	INT		Width in cm

```
CREATE TABLE products (  
  product_id VARCHAR(50) PRIMARY KEY,  
  product_category_name VARCHAR(100),  
  product_name_lenght INT,  
  product_description_lenght INT,  
  product_photos_qty INT,  
  product_weight_g INT,  
  product_length_cm INT,  
  product_height_cm INT,  
  product_width_cm INT  
);
```

orders

Order headers with status and timestamps.

Column	Type	Key	Description
order_id	VARCHAR(50)	PK	Unique order identifier
customer_id	VARCHAR(50)	FK	Links to <code>users.customer_id</code>
order_status	VARCHAR(50)		Status: <code>delivered</code> , <code>shipped</code> , <code>cancelled</code> , <code>created</code> , <code>approved</code>
order_purchase_timestamp	DATETIME		When order was placed
order_approved_at	DATETIME		When payment was approved
order_delivered_carrier_date	DATETIME		When handed to carrier
order_delivered_customer_date	DATETIME		When delivered to customer
order_estimated_delivery_date	DATETIME		Expected delivery date

```
CREATE TABLE orders (  
  order_id VARCHAR(50) PRIMARY KEY,  
  customer_id VARCHAR(50),  
  order_status VARCHAR(50),  
  order_purchase_timestamp DATETIME,
```

```
order_approved_at DATETIME,  
order_delivered_carrier_date DATETIME,  
order_delivered_customer_date DATETIME,  
order_estimated_delivery_date DATETIME,  
FOREIGN KEY (customer_id) REFERENCES users(customer_id)  
);
```

order_items

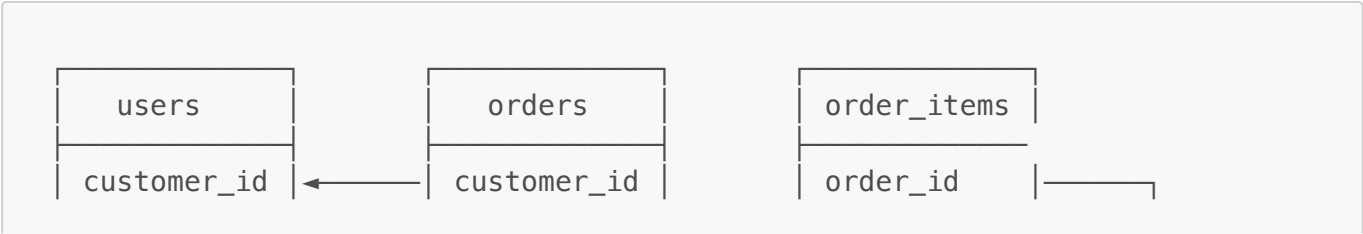
Individual line items within each order.

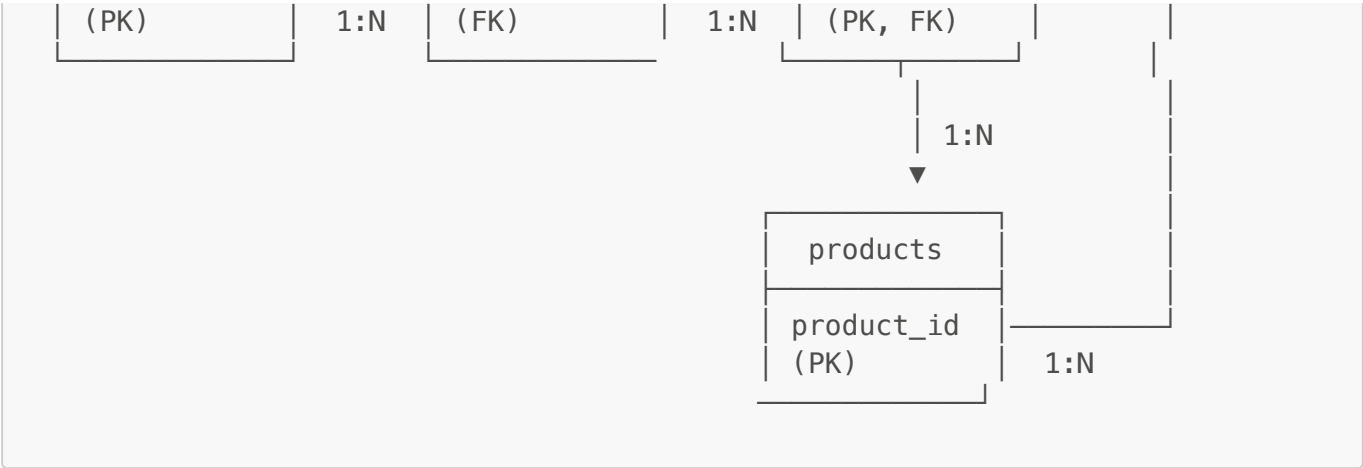
Column	Type	Key	Description
order_id	VARCHAR(50)	PK, FK	Links to orders.order_id
order_item_id	INT	PK	Line item number (1, 2, 3...)
product_id	VARCHAR(50)	FK	Links to products.product_id
seller_id	VARCHAR(50)		Seller identifier
shipping_limit_date	DATETIME		Deadline to ship
price	DECIMAL(10,2)		Item price
freight_value	DECIMAL(10,2)		Shipping cost

```
CREATE TABLE order_items (  
  order_id VARCHAR(50),  
  order_item_id INT,  
  product_id VARCHAR(50),  
  seller_id VARCHAR(50),  
  shipping_limit_date DATETIME,  
  price DECIMAL(10,2),  
  freight_value DECIMAL(10,2),  
  PRIMARY KEY (order_id, order_item_id),  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

2. Table Relationships

Entity Relationship Diagram (Text)





Join Paths

From Table	To Table	Join Condition
users	orders	users.customer_id = orders.customer_id
orders	order_items	orders.order_id = order_items.order_id
products	order_items	products.product_id = order_items.product_id

Standard 4-Table Join Pattern

```
SELECT *
FROM users u
JOIN orders o ON u.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
```

3. Business Term Definitions

Term	Definition	SQL Formula
Revenue	Total sales including shipping	SUM(oi.price + oi.freight_value)
Net Revenue	Sales excluding shipping	SUM(oi.price)
Shipping Cost	Total freight charges	SUM(oi.freight_value)
Best Customer	Highest total spending	SUM(price + freight_value) GROUP BY customer_id ORDER BY ... DESC LIMIT 1
Most Active Customer	Most orders placed	COUNT(DISTINCT order_id) GROUP BY customer_id ORDER BY ... DESC LIMIT 1

Term	Definition	SQL Formula
Last Week	Last 7 days from now	<code>WHERE order_purchase_timestamp >= DATE_SUB(NOW(), INTERVAL 7 DAY)</code>
Last Month	Last 30 days from now	<code>WHERE order_purchase_timestamp >= DATE_SUB(NOW(), INTERVAL 30 DAY)</code>
Completed Order	Successfully delivered	<code>WHERE order_status = 'delivered'</code>
Cancelled Order	Order was cancelled	<code>WHERE order_status = 'cancelled'</code>
Average Order Value (AOV)	Revenue per order	<code>SUM(price + freight_value) / COUNT(DISTINCT order_id)</code>
Items Per Order	Average line items	<code>COUNT(order_item_id) / COUNT(DISTINCT order_id)</code>
New Customer	First-time buyer	Use <code>customer_unique_id</code> to track across orders
Returning Customer	Repeat buyer	<code>customer_unique_id</code> appears in multiple orders

4. Example Queries (Few-Shot Learning)

Q: "How many customers did we gain last week?"

```
SELECT COUNT(DISTINCT customer_id) as new_customers
FROM orders
WHERE order_purchase_timestamp >= DATE_SUB(NOW(), INTERVAL 7 DAY);
```

Q: "Who is our best customer by revenue?"

```
SELECT u.customer_id, u.customer_city, u.customer_state,
       SUM(oi.price + oi.freight_value) as total_spent
FROM users u
JOIN orders o ON u.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY u.customer_id, u.customer_city, u.customer_state
ORDER BY total_spent DESC
LIMIT 1;
```

Q: "What is the most popular product category?"

```
SELECT p.product_category_name, COUNT(oi.order_item_id) as times_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY times_sold DESC
LIMIT 1;
```

Q: "Show me total revenue by state"

```
SELECT u.customer_state,
       SUM(oi.price + oi.freight_value) as total_revenue,
       COUNT(DISTINCT o.order_id) as total_orders
FROM users u
JOIN orders o ON u.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY u.customer_state
ORDER BY total_revenue DESC;
```

Q: "What percentage of orders were delivered on time?"

```
SELECT
    COUNT(CASE WHEN order_delivered_customer_date <=
order_estimated_delivery_date THEN 1 END) * 100.0 / COUNT(*) as
on_time_percentage
FROM orders
WHERE order_status = 'delivered'
    AND order_delivered_customer_date IS NOT NULL
    AND order_estimated_delivery_date IS NOT NULL;
```

5. Security & Generation Rules

Mandatory Rules for SQL Generation

1. **READ-ONLY:** Only generate **SELECT** queries.
2. **NO DESTRUCTIVE OPS:** Never generate **DROP**, **DELETE**, **UPDATE**, **INSERT**, **ALTER**, **TRUNCATE**.
3. **LIMIT RESULTS:** Always add **LIMIT 1000** unless user specifies otherwise.
4. **DATE SAFETY:** Use **order_purchase_timestamp** for date-based filters.
5. **NO COMMENTS:** Do not include **--** or **/* */** in generated SQL.
6. **NO MULTIPLE STATEMENTS:** Only one query per request (no **;** in middle).
7. **USE EXPLICIT JOINS:** Always use **JOIN ... ON** syntax, not implicit joins.
8. **ALIAS TABLES:** Use short aliases (**u**, **o**, **oi**, **p**) for readability.

Ambiguity Handling

If the user query contains ambiguous terms, **do not guess**. Instead, return a clarification request:

Ambiguous Term	Clarification Question	Options
"Best customer"	"How should we define 'best'?"	Revenue, Orders, Retention
"Recent"	"What time period?"	7 days, 30 days, 90 days
"Popular product"	"Measure by what?"	Quantity, Revenue, Order Count
"Sales"	"Include shipping?"	With freight, Without freight

6. Common Query Patterns

Time-Based Filters

```
-- Last 7 days
WHERE order_purchase_timestamp >= DATE_SUB(NOW(), INTERVAL 7 DAY)

-- Last 30 days
WHERE order_purchase_timestamp >= DATE_SUB(NOW(), INTERVAL 30 DAY)

-- Specific date range
WHERE order_purchase_timestamp BETWEEN '2024-01-01' AND '2024-01-31'

-- Year to date
WHERE YEAR(order_purchase_timestamp) = YEAR(NOW())
```

Aggregation Patterns

```
-- Count distinct customers
COUNT(DISTINCT u.customer_unique_id)

-- Count total orders
COUNT(DISTINCT o.order_id)

-- Sum revenue
SUM(oi.price + oi.freight_value)

-- Average per order
AVG(oi.price + oi.freight_value)
```

Status Filters

```
-- Only completed orders
WHERE o.order_status = 'delivered'

-- Exclude cancelled
```

```
WHERE o.order_status NOT IN ('cancelled', 'unsuccessful')

-- Pending orders
WHERE o.order_status IN ('created', 'approved', 'processing')
```

7. 📊 Database Statistics (Optional Context)

Table	Approx. Rows	Last Updated
users	~100,000	2018
products	~33,000	2018
orders	~100,000	2018
order_items	~113,000	2018

Note: This is historical data from 2017-2018. Do not use for real-time analytics.

8. 🚫 Common Mistakes to Avoid

Mistake	Correction
Using <code>users.customer_unique_id</code> for joins	Use <code>users.customer_id</code> for joining to orders
Forgetting to include <code>freight_value</code> in revenue	Revenue = <code>price + freight_value</code>
Using <code>order_items</code> without joining <code>orders</code>	Always join through <code>orders</code> to get customer info
Assuming one customer = one <code>customer_id</code>	One person can have multiple <code>customer_id</code> values
Not filtering by <code>order_status</code>	Consider filtering for <code>'delivered'</code> for accurate metrics
Using <code>NOW()</code> for historical data	Data is from 2017-2018, use specific dates for accuracy

End of RAG Context Document